

13

Neural Networks

The brain-inspired models behind modern AI

Where Deep Learning begins

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A model loosely inspired by how the brain works
Made of	Tiny units called neurons, joined in layers
Powers	Image recognition, language models, self-driving, more
Deep Learning	Just a neural network with many layers
Why now	Classical ML can't handle images/text as well as this

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	Why Neural Networks?	What classical ML can't do
2	The Neuron	The basic building block
3	Weights and Bias	The numbers the network learns
4	Activation Functions	Adding the 'bend' (ReLU, sigmoid)
5	Layers and the Network	How neurons join together
6	The Forward Pass	How a prediction is made
7	Loss and Backpropagation	How it learns from mistakes
8	Training Words	Epochs, batches, learning rate
9	A First Network in Code	Full PyTorch-style example
10	Deep Learning	What 'deep' really means
11	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. Why Neural Networks?

The models we learned so far (linear regression, trees, XGBoost) are great for table data. But they struggle with messy, complicated data like images, sound, and language. How do you draw a line or a tree to recognise a cat in a photo? You can't, easily.

Neural networks (■■■■■■■ ■■■■■■■■■■■■) can learn very complicated patterns that simpler models cannot. They are loosely inspired by the human brain, which is made of billions of tiny connected cells called neurons. A neural network copies this idea with simple math units, also called neurons, connected together in layers.

When a neural network has many layers, we call it **deep learning** (■■■ ■■■■■■■■■■). This is the technology behind face recognition, voice assistants, language models like the one writing this, and self-driving cars.

■ *Important: a neural network is NOT a real brain. It's just a lot of simple math arranged in a clever way. The 'brain' comparison is only a loose metaphor to help it make sense.*

2. The Neuron — The Building Block

Everything starts with a single **neuron** (■■■■■■■). A neuron is a tiny unit that does three simple things: it takes some inputs, combines them, and produces one output.

One Neuron: inputs x weights, add them up, then activate

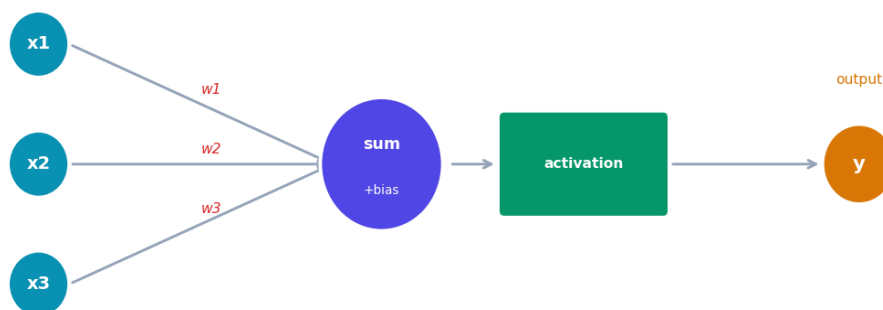


Fig 1 — One neuron: multiply each input by a weight, add them up (plus bias), then apply activation.

Here's what a neuron does, step by step:

- **Take inputs** — numbers coming in (like pixel values, or features).
- **Multiply each by a weight** — a weight says how important that input is.
- **Add them all up**, plus a bias number.
- **Apply an activation** — a function that adds a 'bend' (more on this soon).
- **Output one number** — passed on to the next neurons.

Notice this is almost exactly linear regression ($w \cdot x + b$) with one extra step on the end. A neuron is basically a tiny linear model. The power comes from connecting thousands of them.

5. Layers and the Network

One neuron is weak. The magic happens when you stack many neurons into **layers** (■■■■■■■ — ■■■■) and connect the layers together. Each layer's output becomes the next layer's input.

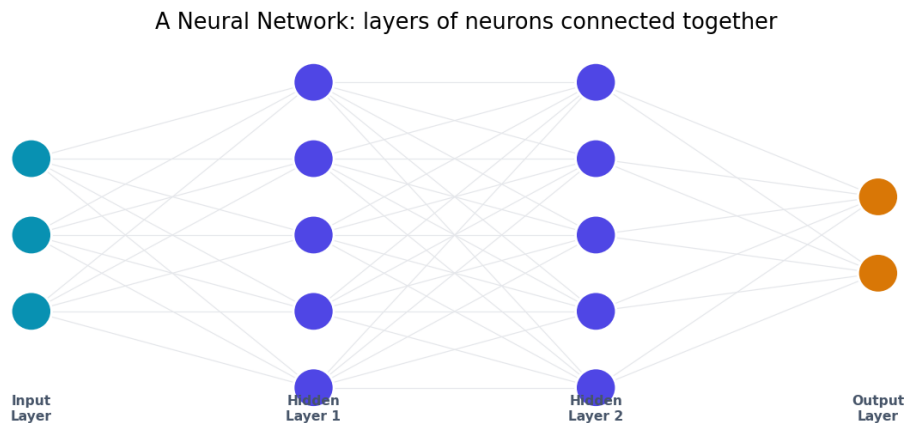


Fig 3 — A neural network. Data enters the input layer, flows through hidden layers, exits the output layer.

Layer	Its job
Input layer	Receives the raw data (one neuron per feature)
Hidden layers	Do the real work — find patterns step by step
Output layer	Gives the final answer (a number or a category)

Each layer learns something more advanced than the one before. In an image network, early layers might detect edges, middle layers detect shapes like eyes and ears, and later layers detect whole faces. The network builds understanding step by step, layer by layer.

6. The Forward Pass — Making a Prediction

The **forward pass** (■■■■■■■■■■ ■■■■) is how the network makes a prediction. Data goes in the front, flows through every layer doing its math, and a prediction comes out the other end. It's just a chain of the neuron operation, layer after layer.

input → layer 1 → layer 2 → ... → output (prediction)

At the very start, with random weights, the forward pass gives a bad, random prediction. That's fine — the network hasn't learned yet. Next we need a way to measure how wrong it was and fix it.

7. Loss and Backpropagation — How It Learns

After the forward pass gives a prediction, we measure how wrong it is using a **loss function** (■■ ■■■■■■ — ■■■■■ ■■■■■■) — the same idea as the cost function from linear regression. A big loss means a bad prediction.

Then comes the clever part: **backpropagation** (■■■■■■■■■■■■■■■■■■■■). The error is sent **backward** through the network, from the output toward the input. Along the way, it works out how much each weight contributed to the mistake, and nudges every weight a little to reduce the error. This uses gradient descent — the same downhill idea from before.

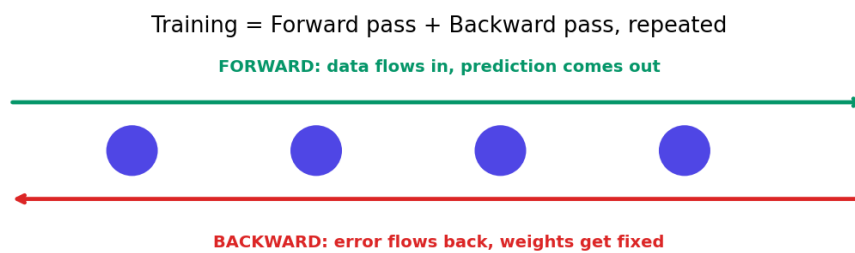


Fig 4 — Training repeats two steps: forward (predict) then backward (fix the weights).

So training is just these two steps, repeated thousands of times:

- **Forward pass:** make a prediction with the current weights.
- **Backward pass:** measure the error, send it back, and adjust every weight.

Each round, the predictions get a tiny bit better. After many rounds, the network's millions of weights have settled into values that produce good answers. That's how a neural network learns.

■ *You don't compute backpropagation by hand. Libraries like PyTorch and TensorFlow do all the calculus for you automatically. You just describe the network and the data.*

8. Training Words You'll Hear

Word	Plain meaning
Epoch	One full pass through ALL the training data
Batch	A small chunk of data processed at once
Batch size	How many examples in each chunk (e.g. 32)
Learning rate	How big each weight adjustment is (the step size)
Loss	How wrong the network currently is (lower is better)
Gradient	The direction to nudge weights to reduce loss
Optimizer	The method that updates weights (e.g. Adam, SGD)

■ *A typical training run does many epochs. Within each epoch, the data is split into batches, and the weights update once per batch. You watch the loss drop epoch by epoch — that's the network learning.*

9. A First Network in Code

Here is a tiny neural network in PyTorch (the deep learning library coming up next topic). Don't worry about every detail — just see the shape of it.

```
import torch
import torch.nn as nn

# Define a simple network: 4 inputs -> 16 hidden -> 3 outputs
model = nn.Sequential(
    nn.Linear(4, 16),    # layer 1: 4 inputs to 16 neurons
    nn.ReLU(),           # activation (the bend)
    nn.Linear(16, 3)     # output layer: 16 to 3 classes
)

# The two tools for learning
loss_fn = nn.CrossEntropyLoss()    # measures error
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# The training loop (forward + backward, repeated)
for epoch in range(100):
    preds = model(X)                # forward pass
    loss = loss_fn(preds, y)         # how wrong are we?

    optimizer.zero_grad()           # reset old gradients
    loss.backward()                 # backward pass (backprop)
    optimizer.step()                # update the weights

    if epoch % 20 == 0:
        print(f'Epoch {epoch}, Loss: {loss.item():.3f}')
```

■ See the pattern? forward → measure loss → backward → update. That four-line loop is the heart of training every neural network, from this tiny one to giant language models.

10. What 'Deep Learning' Really Means

Deep learning simply means a neural network with **many hidden layers** — that's the whole secret behind the fancy name. 'Deep' = many layers stacked up.

More layers let the network learn more complex things. A shallow network might tell cats from dogs. A deep network with hundreds of layers can describe a photo in a sentence, translate languages, or hold a conversation. The trade-off: deeper networks need much more data and computing power (often GPUs) to train.

Type	Layers	Can handle
Simple neural net	1-2 hidden	Basic patterns, table data
Deep network	Many hidden	Images, audio, complex patterns

Very deep (modern AI)	Dozens to hundreds	Language, vision, generation
-----------------------	--------------------	------------------------------

11. Common Mistakes

Mistake	Why it's a problem	Fix
Not scaling inputs	Network trains badly or not at all	Scale features first
Learning rate too high	Loss jumps around, never settles	Lower it (try 0.001)
Too few epochs	Network hasn't finished learning	Train longer, watch the loss
Too big a network for little data	Overfits — memorises	Use a smaller net or more data
Forgetting zero_grad()	Gradients pile up wrongly	Call optimizer.zero_grad() each step
No activation function	Network can only do straight lines	Add ReLU between layers

Quick Reference Cheatsheet

Concept	In one line
Neuron	inputs x weights, summed, +bias, then activated
Weight	how important an input is (network learns it)
Bias	shifts the result up or down
Activation	adds the bend; ReLU is the default
Layer	a group of neurons; input / hidden / output
Forward pass	data flows in, prediction comes out
Loss	how wrong the prediction is
Backpropagation	send error back, fix every weight
Epoch	one full pass over all training data
Learning rate	step size for adjusting weights
Deep learning	a neural network with many layers

The big picture to remember: a neural network is many simple neurons in layers. It predicts with a forward pass, measures its error, then uses backpropagation to fix its weights — over and over until it gets good.

Next Topic: PyTorch — the most popular library for actually building and training neural networks. We'll turn everything from this topic into real, runnable code.